



Module 2 — Architecture

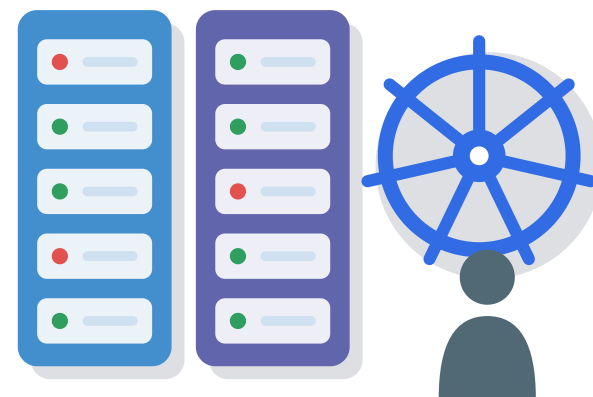
Control plane, nœuds et parcours d'une requête

Kubernetes — Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous saurez :

- nommer les composants du **control plane** et leur rôle ;
- expliquer pourquoi l'**API Server** est le seul à parler à **etcd** ;
- décrire le travail du **kubelet**, de **kube-proxy** et du runtime ;
- suivre un `kubectl apply` jusqu'au conteneur démarré ;
- expliquer la **boucle de réconciliation**.

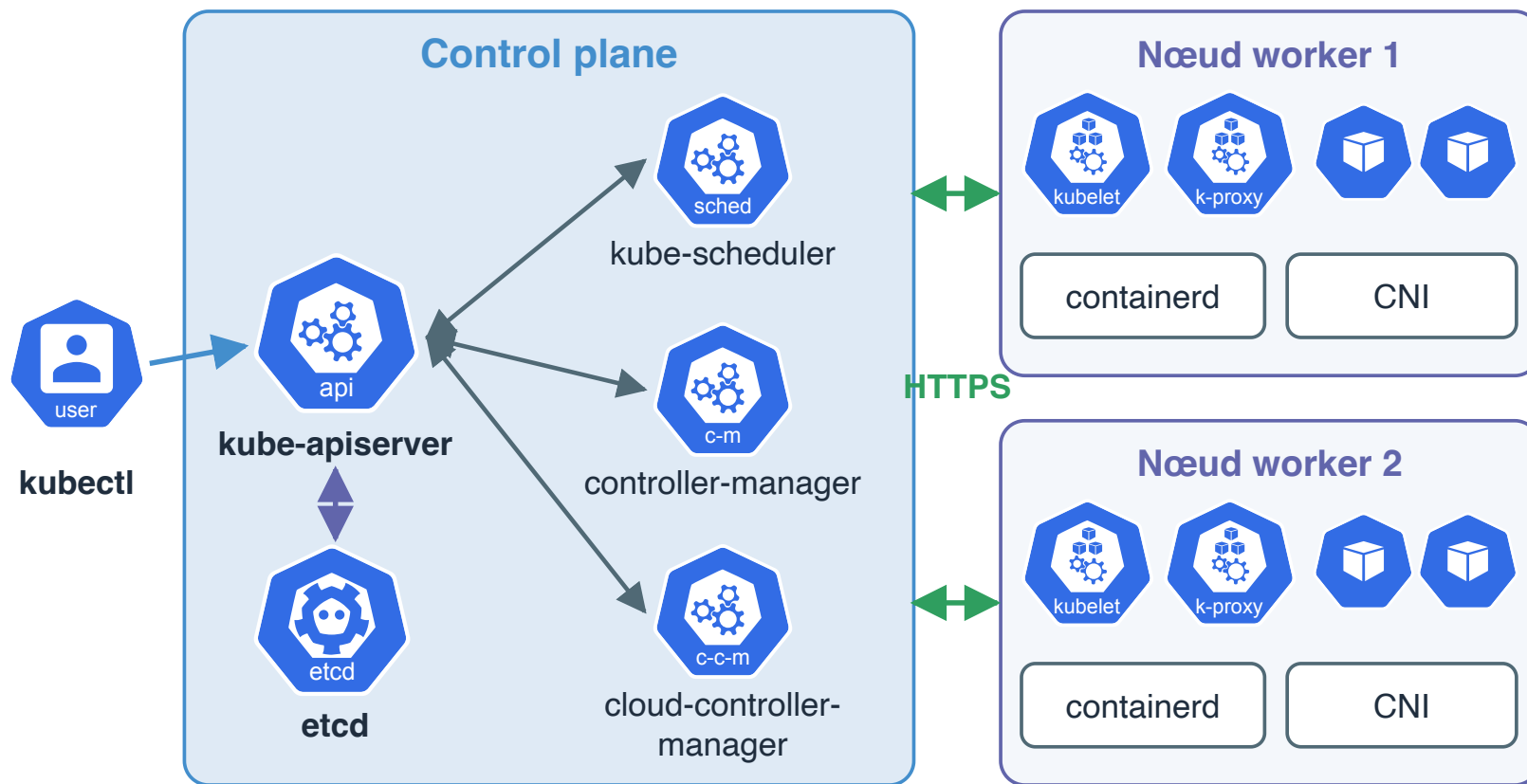


Plan du module

1. **Vue d'ensemble** : control plane et nœuds
 2. **Le control plane** : API Server, etcd, scheduler, controller-manager, cloud-controller-manager
 3. **Les nœuds** : kubelet, static Pods, runtime CRI, kube-proxy, CNI
 4. **De `kubect1 apply` au conteneur** : séquence, chronologie mesurée, watch, haute disponibilité
- Puis la **Démo 2** (observer les composants) et le **TP 2** (explorer l'architecture de TrackR).

1. Vue d'ensemble

Deux familles de composants



Sur le lab : le control plane est fait de Pods

```
$ kubectl get pods -n kube-system \
  -o custom-columns=NOM:.metadata.name,IP:.status.podIP,NOEUD:.spec.nodeName
```

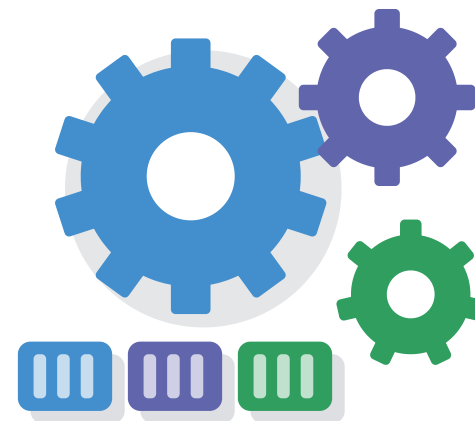
NOM	IP	NOEUD
coredns-7d764666f9-vdwcd	10.244.0.3	k8s-formation-control-plane
etcd-k8s-formation-control-plane	10.89.2.5	k8s-formation-control-plane
kindnet-mkpmj	10.89.2.4	k8s-formation-worker
kube-apiserver-k8s-formation-control-plane	10.89.2.5	k8s-formation-control-plane
kube-controller-manager-k8s-formation-control-plane	10.89.2.5	k8s-formation-control-plane
kube-proxy-4hqhg	10.89.2.4	k8s-formation-worker
kube-scheduler-k8s-formation-control-plane	10.89.2.5	k8s-formation-control-plane
...		

Le control plane **décide**, les nœuds **exécutent**. Les composants système partagent l'**IP du nœud** grâce à `hostNetwork: true` ; CoreDNS a une IP de Pod.

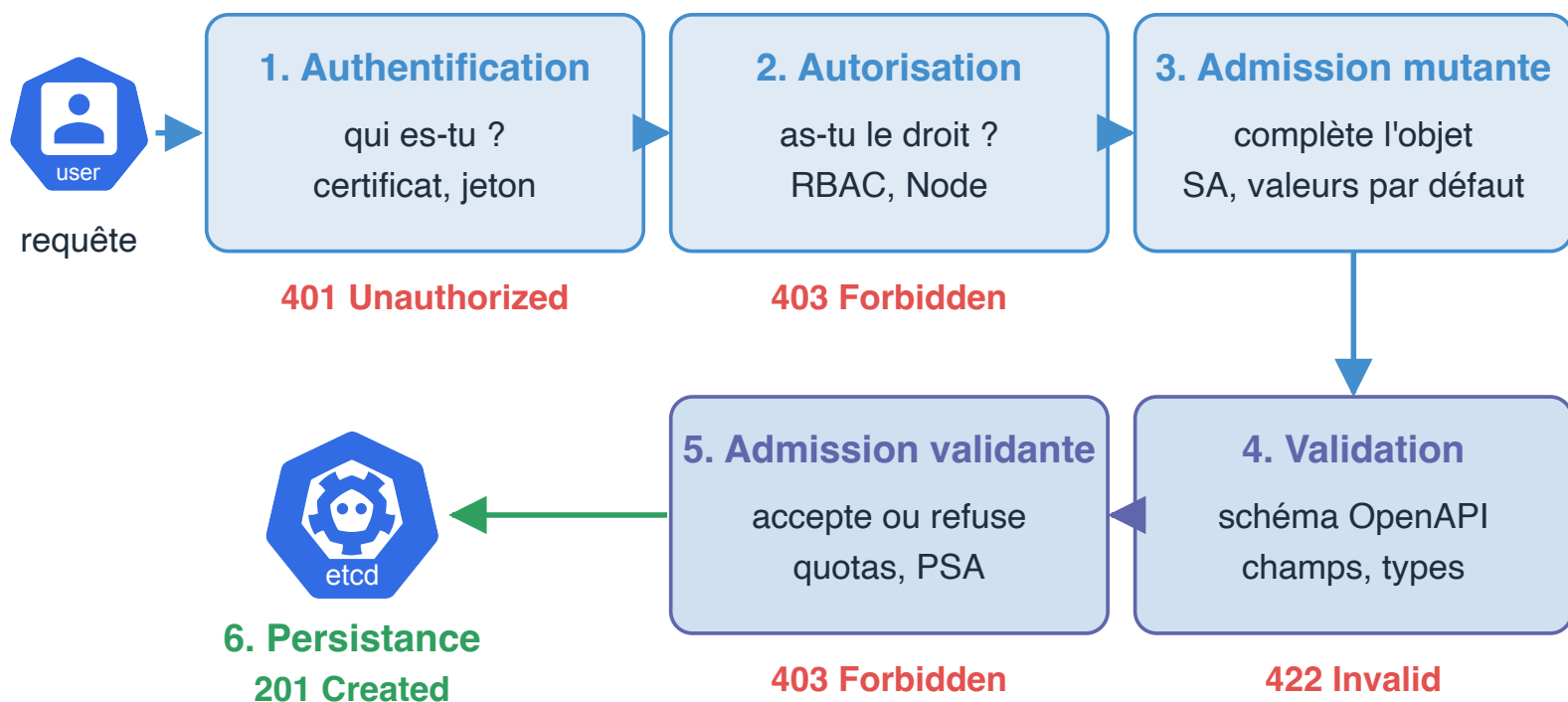
2. Le control plane

L'API Server, point d'entrée unique

- **Toutes** les interactions passent par lui : `kubectl`, contrôleurs, scheduler, kubelets.
- Une **API REST** sur HTTPS (port 6443) : chaque ressource a une URL.
- Il **authentifie**, **autorise**, passe l'**admission**, puis **valide**.
- C'est le **seul** composant qui lit et écrit dans etcd.
- Il est **sans état** : on peut en lancer plusieurs derrière un répartiteur.



Le pipeline d'une requête



Une API REST : `kubectl -v=6`

`kubectl apply -f trackr-api-deployment.yaml -n trackr -v=6` révèle les appels HTTP :

```
GET /openapi/v3/apis/apps/v1 200 OK
GET /apis/apps/v1/namespaces/trackr/deployments/trackr-api 404 Not Found
GET /api/v1/namespaces/trackr 200 OK
POST /apis/apps/v1/namespaces/trackr/deployments?... 201 Created
```

- `kubectl` cherche d'abord l'objet : **404** signifie « à créer ».
- La création est un simple **POST** ; une mise à l'échelle est un **PATCH** sur `/scale`.
- Tout ce que fait `kubectl`, un `curl` muni d'un certificat peut le faire.

Authentification, autorisation, admission

```
$ kubectl auth whoami
ATTRIBUTE                                VALUE
Username                                kubernetes-admin
Groups                                  [kubeadm:cluster-admins system:authenticated]
...

$ kubectl get pods -n trackr --as=system:serviceaccount:trackr:default
Error from server (Forbidden): pods is forbidden:
User "system:serviceaccount:trackr:default" cannot list resource "pods"
in API group "" in the namespace "trackr"
```

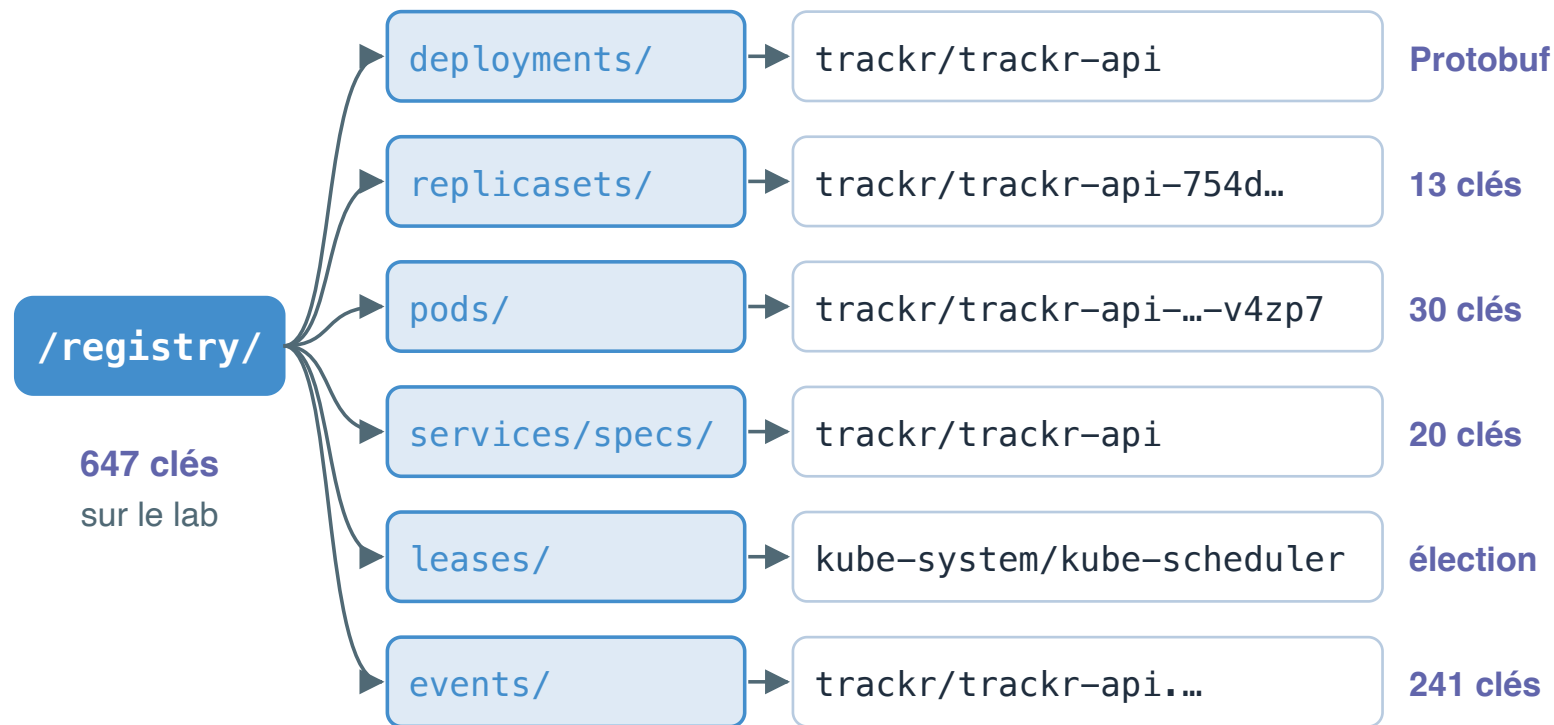
- **Qui ?** un certificat X.509 signé par la CA du cluster.
- **Le droit ?** `--authorization-mode=Node, RBAC` : le compte de service n'a aucun droit.
- **Admission** : nos 27 lignes de YAML deviennent un objet de 69 lignes (jeton de ServiceAccount injecté, valeurs par défaut).

etcd, la mémoire du cluster

- Base **clé/valeur** distribuée, cohérente (CNCF, écrite en Go).
- Stocke **toute** la configuration et l'état : objets, Secrets, statuts, événements.
- Valeurs encodées en **Protobuf** sous
`/registry/<type>/<namespace>/<nom>`.
- Accès en **TLS mutuel** uniquement, par l'API Server.
- Perdre etcd sans sauvegarde, c'est perdre le cluster.



L'arborescence `/registry`



Mesuré sur le lab, kind v1.35, 3 nœuds (décomptes de tout le cluster)

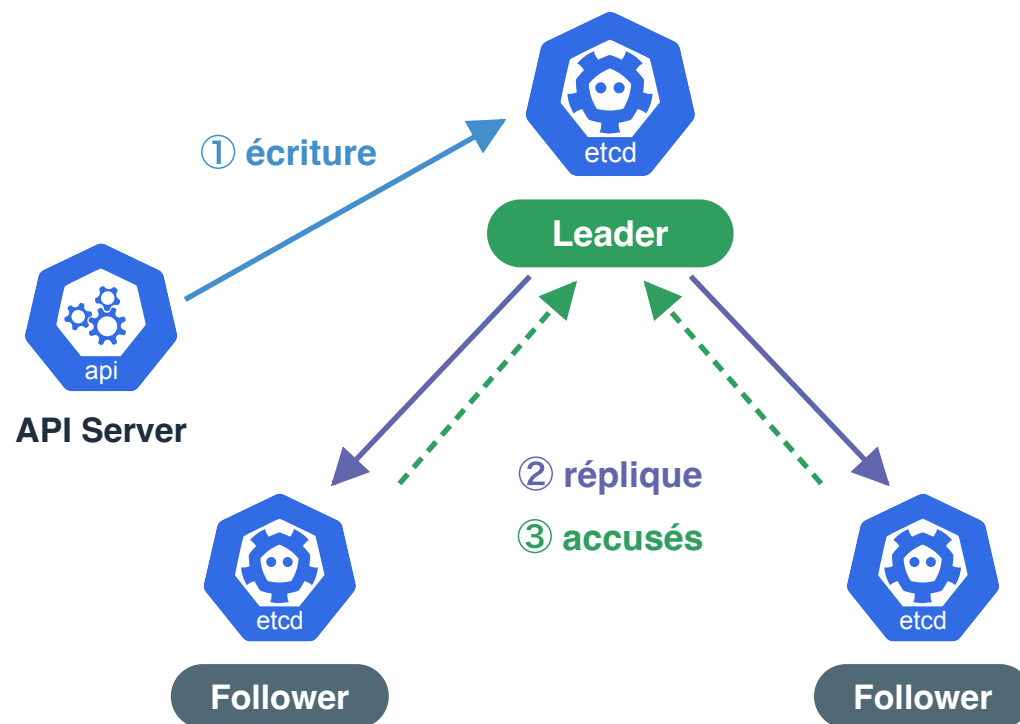
Lire etcd avec etcdctl

```
$ kubectl -n kube-system exec etcd-k8s-formation-control-plane -- etcdctl \
  --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/deployments/trackr --prefix --keys-only
/registry/deployments/trackr/trackr-api

$ etcdctl ... get /registry/deployments/trackr/trackr-api -w json # métadonnées
create_revision: 2182 mod_revision: 2420 version: 6
```

- `version: 6` : la clé a été réécrite **6 fois** en une minute (statut mis à jour par le contrôleur).
- La valeur brute est binaire : on lit les objets avec `kubectl`, pas avec `etcdctl`.

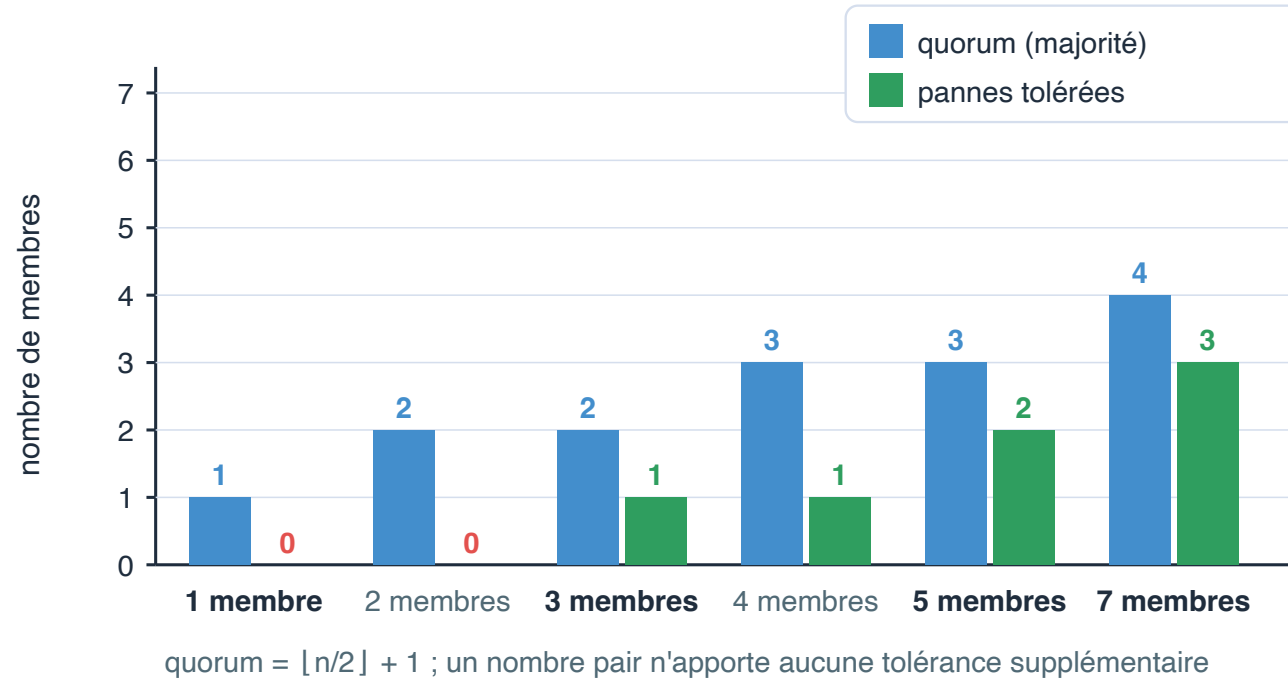
Raft : un leader, des followers



Une écriture

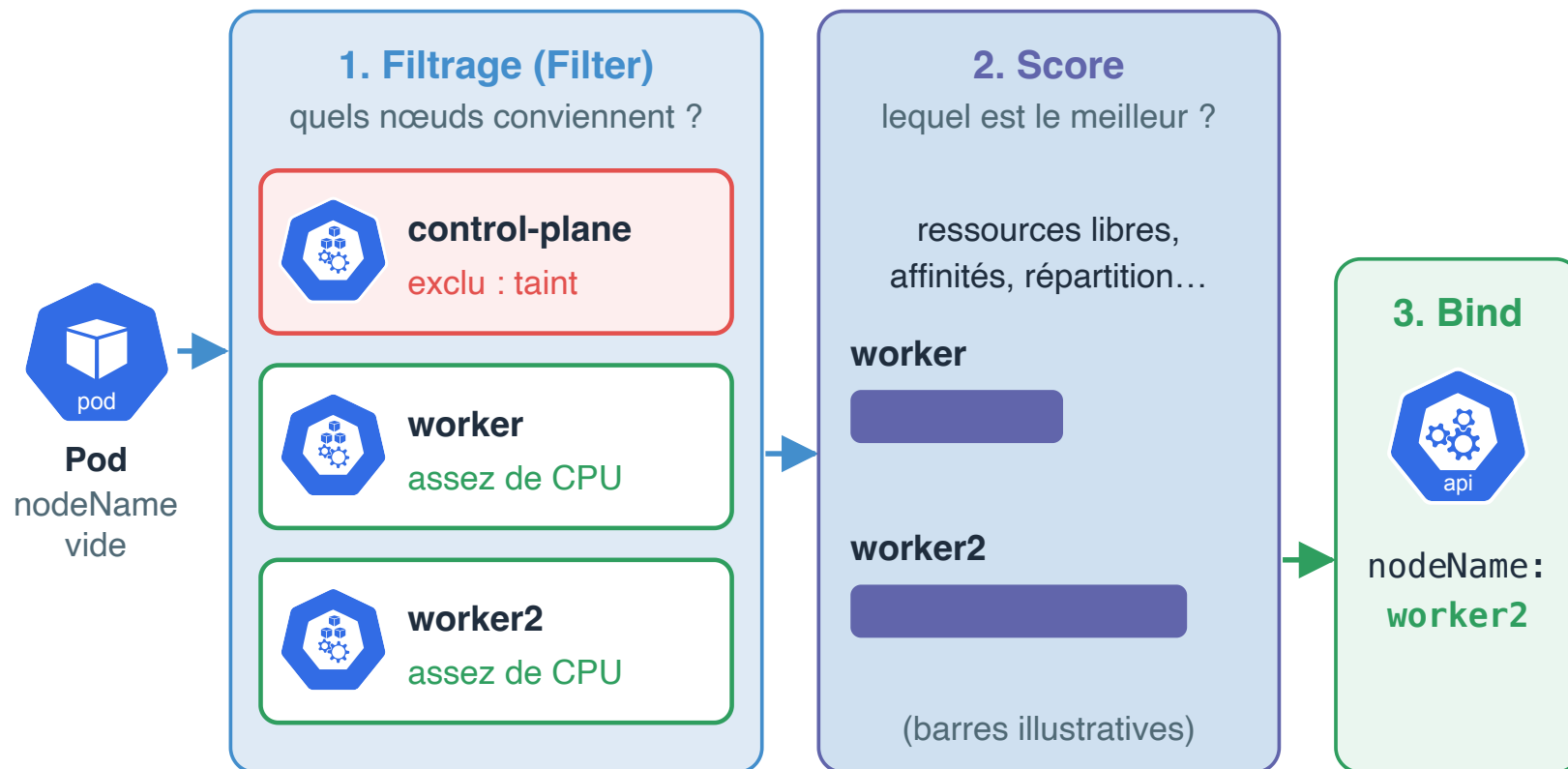
- ① arrive au leader
- ② copiée dans le journal des followers
- ③ validée dès que la majorité l'a écrite
- ④ réponse au client

Quorum et tolérance aux pannes



Source : FAQ etcd (fault tolerance) ; le lab n'a qu'1 membre : 0 panne tolérée

Le scheduler : filtrer puis noter



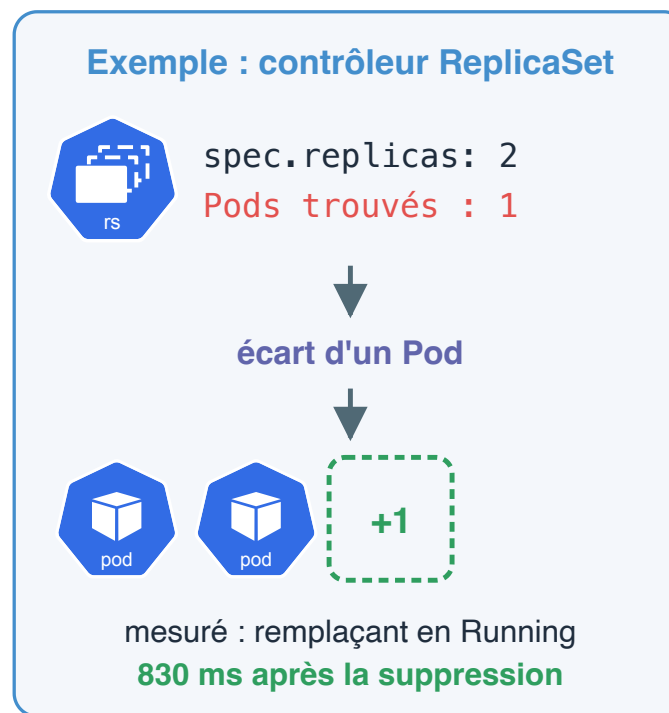
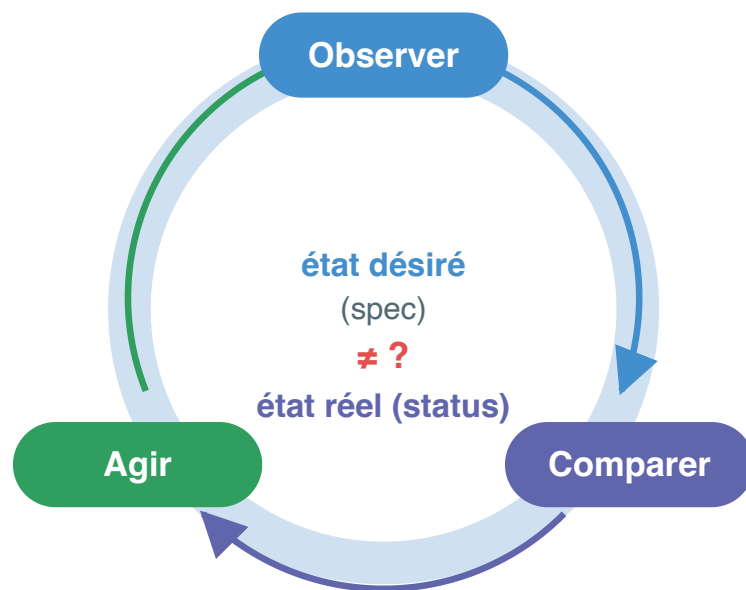
Quand le filtrage échoue

Un Pod `trackr-api-gourmand` qui demande `cpu: "64"` reste en `Pending` :

```
$ kubectl describe pod trackr-api-gourmand -n trackr | sed -n '/Events:/,$p'  
Warning FailedScheduling default-scheduler 0/3 nodes are available:  
  1 node(s) had untolerated taint(s), 2 Insufficient cpu.  
  preemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.
```

- Le message **est** le compte rendu du filtrage : 1 nœud exclu par taint, 2 par manque de CPU.
- Le scheduler ne lance rien : il écrit seulement `spec.nodeName` (le **binding**).

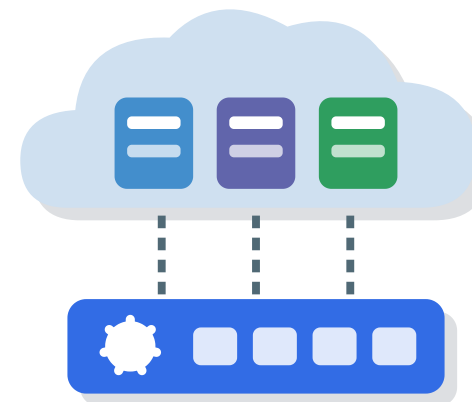
Le controller-manager : réconcilier



kube-controller-manager regroupe les contrôleurs natifs : Deployment, ReplicaSet, Node, Job...

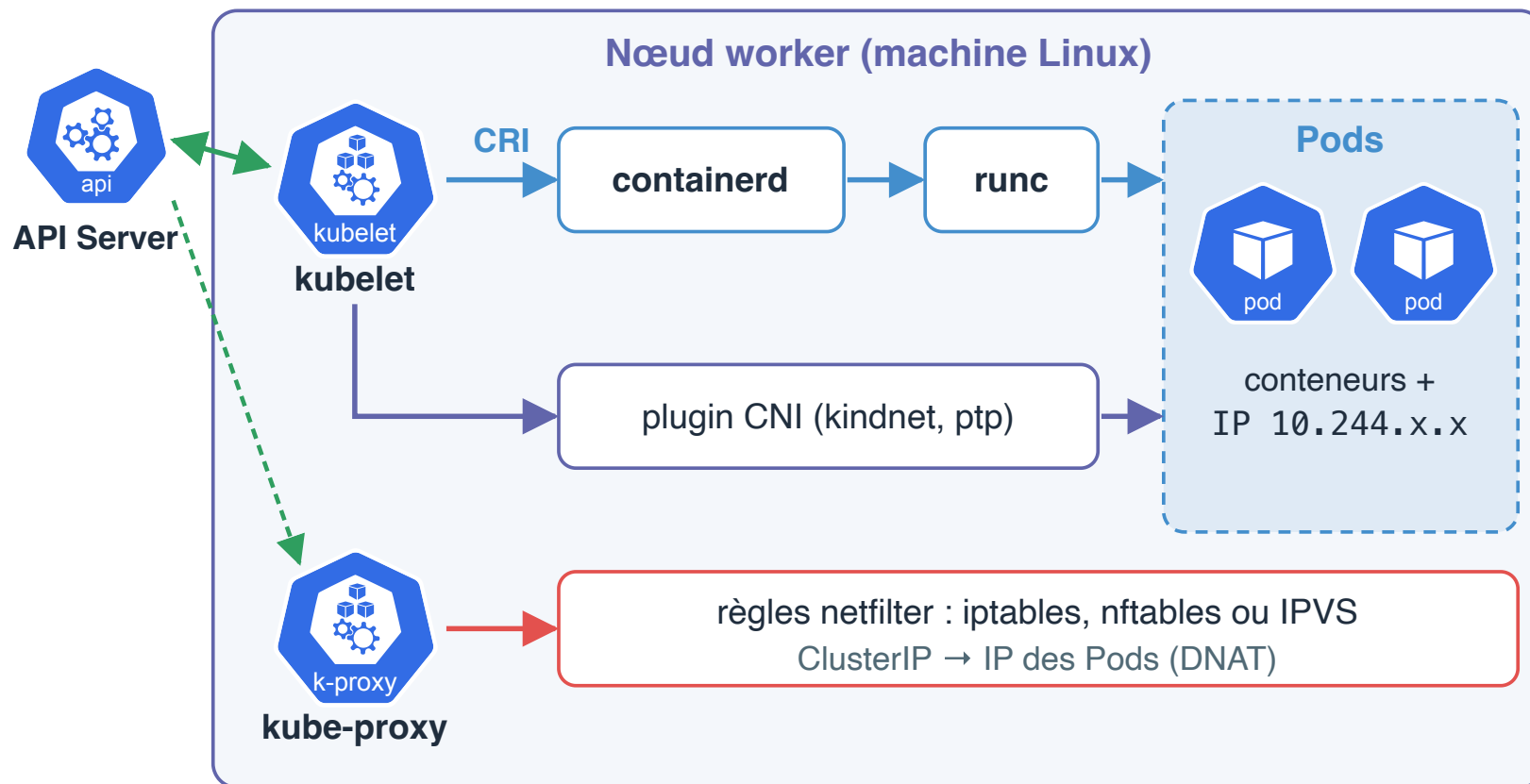
cloud-controller-manager

- Isole le code **propre à un fournisseur** cloud du cœur de Kubernetes.
- **Node** : vérifie qu'une VM existe encore, ajoute zone et type d'instance.
- **Route** : programme les routes du réseau du cloud.
- **Service** : crée un vrai répartiteur de charge pour `type: LoadBalancer`.
- Absent du lab kind (non rejoué sur le lab de formation).



3. Les nœuds

Anatomie d'un nœud worker

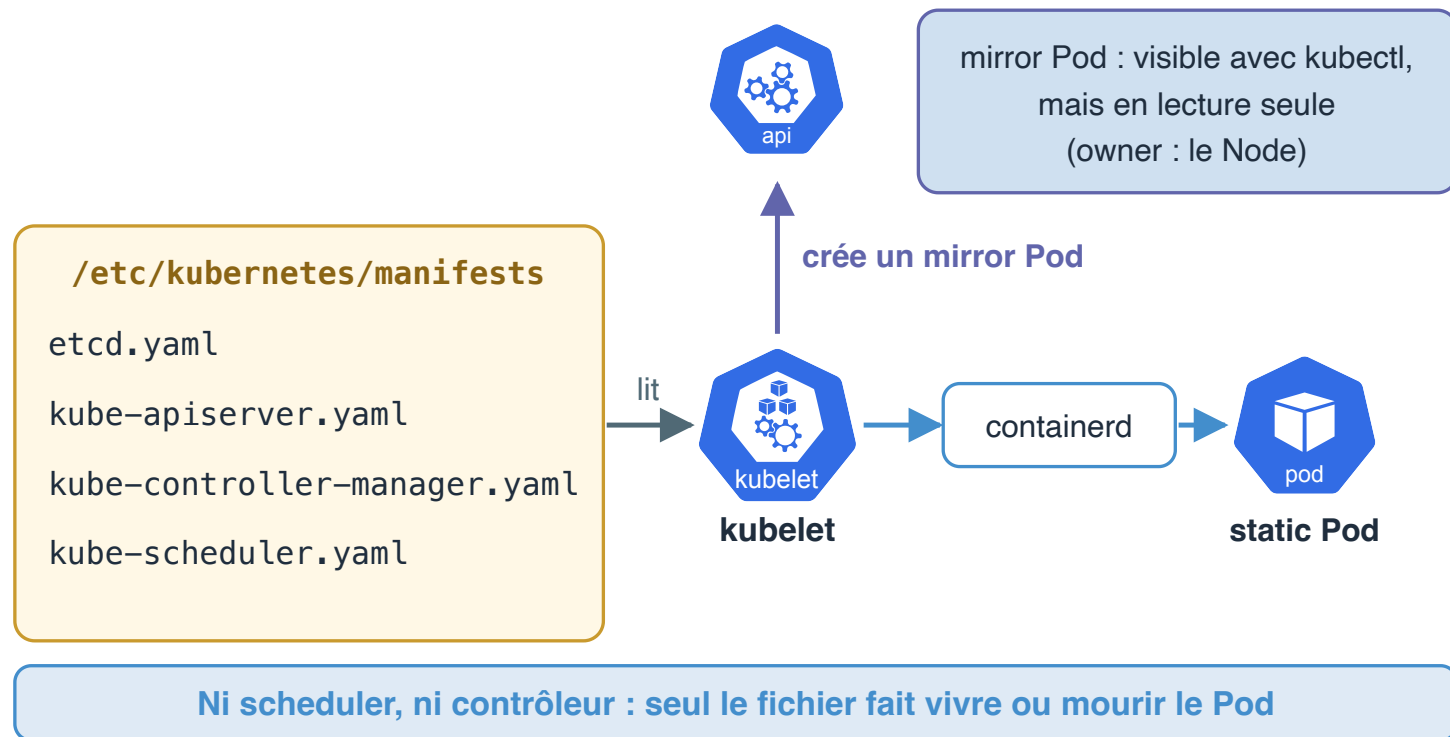


Le kubelet, agent de chaque nœud

```
$ docker exec k8s-formation-worker systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
   Active: active (running) since Sun 2026-10-04 20:37:02 UTC; 13min ago
   Main PID: 197 (kubelet)
   Memory: 36.3M
           └─197 /usr/bin/kubelet --kubeconfig=/etc/kubernetes/kubelet.conf
              --config=/var/lib/kubelet/config.yaml --node-ip=10.89.2.4 ...
```

- Un **service systemd**, pas un Pod : il doit tourner avant tout le reste.
- Il **surveille** les Pods qui lui sont affectés, lance les sondes, remonte leur statut.
- Il **rapporte l'état du nœud** : conditions `Ready`, `MemoryPressure` ... et un **Lease** renouvelé (durée 40 s).

Les static Pods



Testé sur un cluster jetable : supprimer le mirror Pod ne tue pas le conteneur.

Le runtime : CRI et containerd

```
$ docker exec k8s-formation-worker crictl version
RuntimeName: containerd
RuntimeVersion: v2.2.0
RuntimeApiVersion: v1
```

```
$ docker exec k8s-formation-worker crictl ps
```

CONTAINER	STATE	NAME	POD	NAMESPACE
2e1a363bcd367	Running	metrics-server	metrics-server-5db8b86858-brzxf	kube-system
235a9f018a6bc	Running	kindnet-cni	kindnet-mkpmj	kube-system
402a28f04c0dd	Running	kube-proxy	kube-proxy-4hqhg	kube-system

- Le kubelet parle au runtime via la **CRI** (Container Runtime Interface, gRPC).
- `crictl` est l'outil de diagnostic côté nœud (`docker exec` , ou `podman exec` avec Podman).

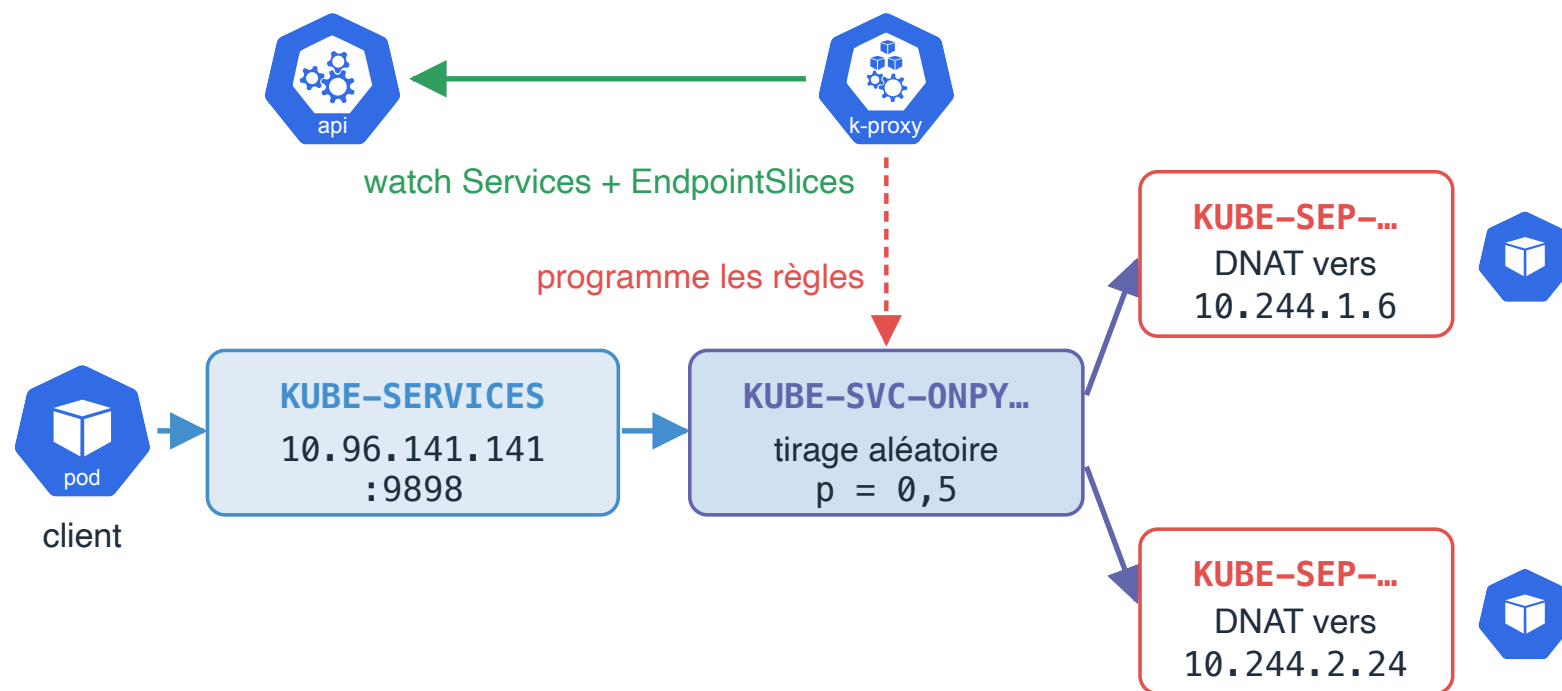
kube-proxy : trois modes

Mode	Mécanisme	Quand l'utiliser
iptables	règles netfilter, choix aléatoire	Défaut, mode du lab
nftables	successeur d'iptables (GA en 1.33)	Noyaux récents, gros clusters
ipvs	répartiteur noyau (IPVS)	Déprécié en 1.35

```
$ kubectl logs -n kube-system kube-proxy-4hqhg | grep Proxier  
"Using iptables Proxier"
```

kube-proxy **ne relaie aucun paquet** : il programme le noyau, qui fait le travail.

Un Service vu par iptables



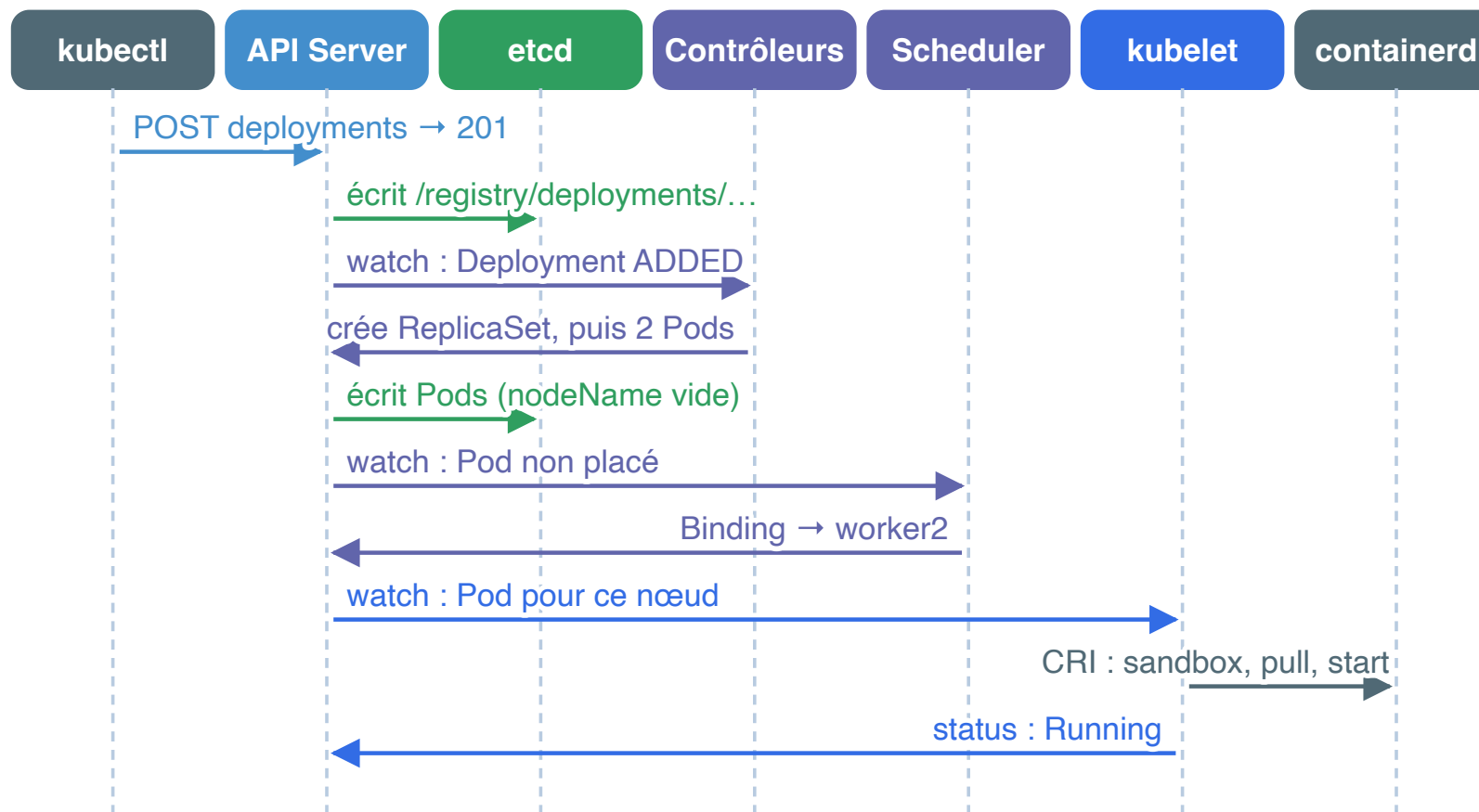
Règles relevées sur le nœud worker du lab : iptables-save -t nat

CNI : le réseau des Pods

- Le **CNI** (Container Network Interface) est un standard : le runtime appelle un **plugin** à chaque création de Pod.
- Le plugin crée l'interface réseau et attribue l'**IP du Pod** (ici 10.244.x.x).
- Sur le lab, **kindnet** dépose `/etc/cni/net.d/10-kindnet.conflist` : plugins `ptp`, `host-local` (IPAM) et `portmap`.
- Chaque nœud reçoit un sous-réseau : `10.244.0.0/24` pour le control plane.
- Autres plugins courants : Calico, Cilium, Flannel.

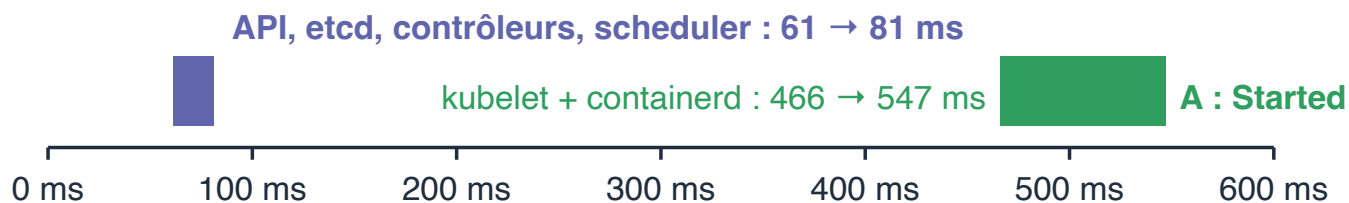
4. De **kubect1 apply** au conteneur

Le parcours complet d'un Deployment

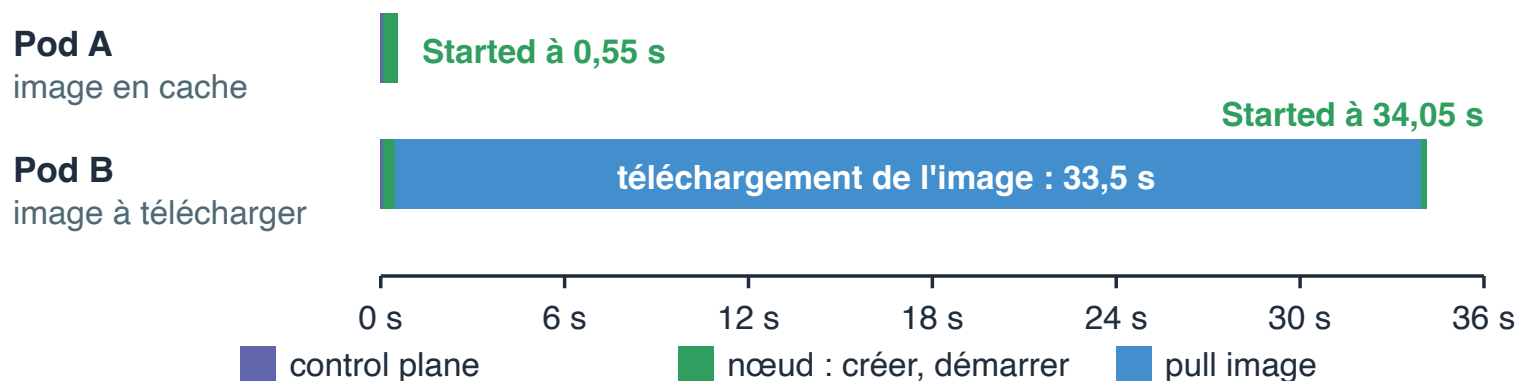


Chronologie mesurée de trackr-api

Zoom sur les 600 premières millisecondes

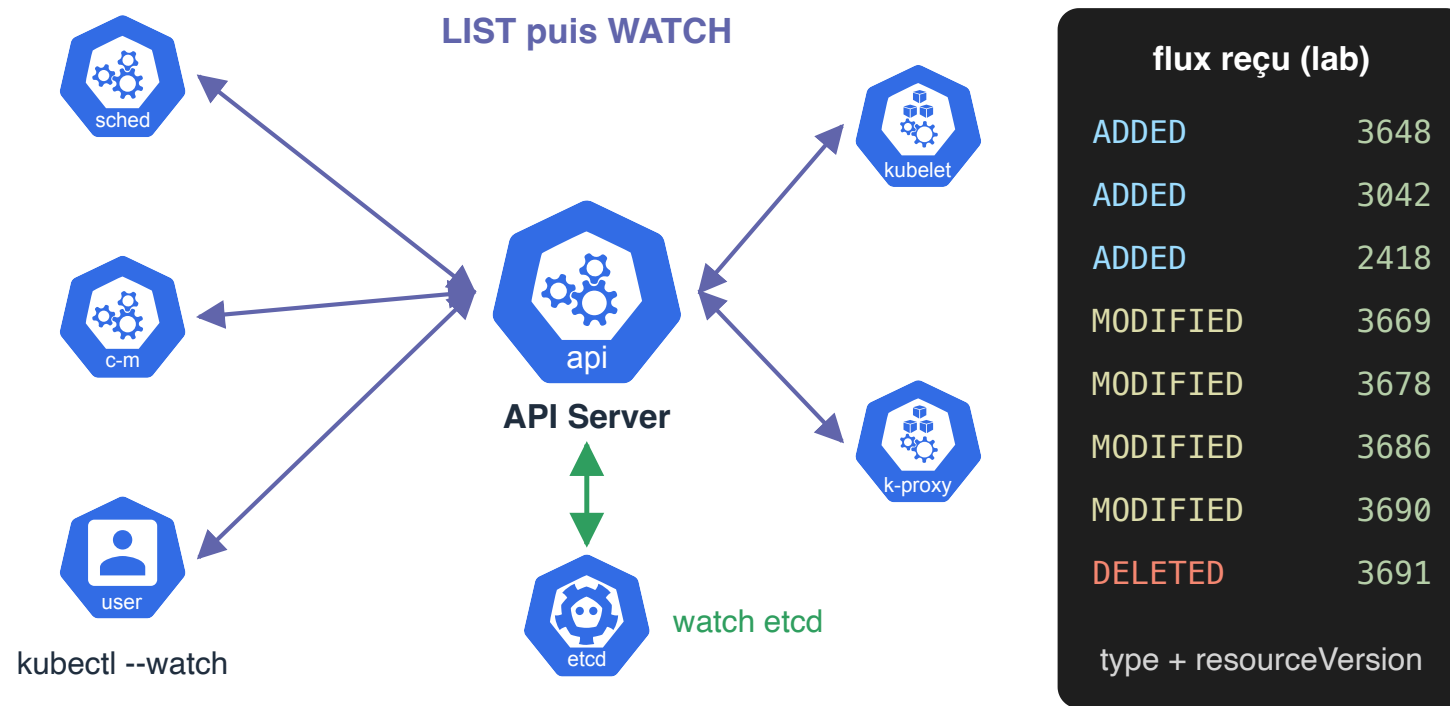


Vue complète (secondes)



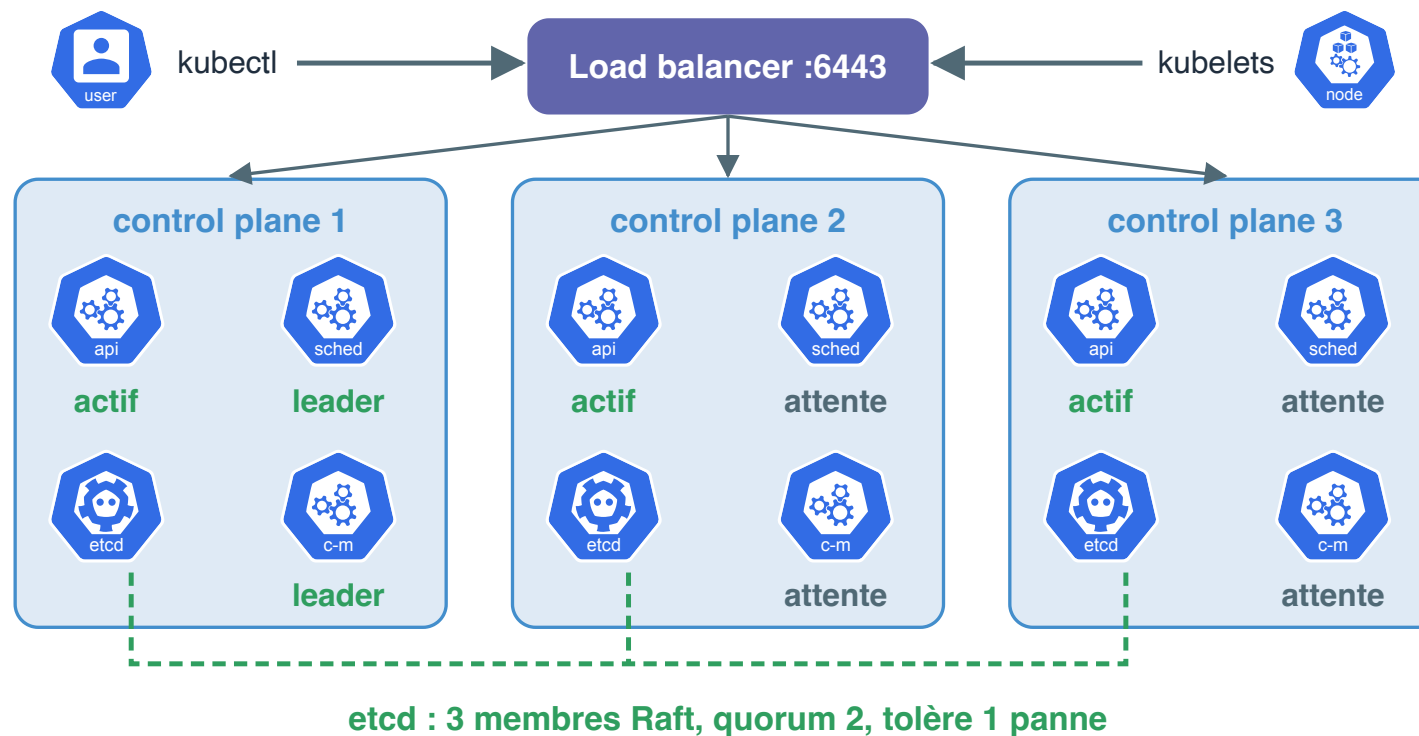
Mesuré sur le lab, kind v1.35, 3 nœuds (événements horodatés pendant un kubectl apply)

Watch : personne n'interroge en boucle



LIST puis requête `?watch=true` gardée ouverte : l'API Server **pousse** les changements.

Haute disponibilité du control plane



API Servers **actifs-actifs** ; scheduler et controller-manager **actif-passif** via un Lease.

Démo 2 et TP 2

Démo 2 – Observer les composants

`demos/demo-02-composants.md`

Static Pods, etcdctl, `crictrl`, kubelet, `/livez`, appels REST et événements d'un déploiement.

TP 2 – Explorer l'architecture

`tp/tp-02-explorer-architecture.md`

Retrouver chaque composant de TrackR sur le lab, lire sa trace dans etcd et mesurer sa chronologie.

À retenir

- Le **control plane décide**, les **nœuds exécutent**.
- L'**API Server** est la porte unique : authentication, autorisation, admission, puis etcd.
- **etcd** garde tout l'état ; Raft impose un **quorum** : 3 ou 5 membres.
- Le **scheduler** filtre puis note ; les **contrôleurs** réconcilient sans fin.
- Le **kubelet** pilote containerd via la CRI ; **kube-proxy** programme le noyau.
- Tout repose sur le **watch** : décision en moins de 100 ms, le reste est le temps du nœud.